element `X` from the temporary variable to the one child of `Y` that belongs to `P` (step 4).

Notice that this method, if the height of the tree is `H`, requires at most `H` swaps, because each time we swap two elements, we move one level up in the tree, towards its root.

This result is an important result because, as we will see in section 2.7, since heaps are balanced binary trees, their height is logarithmic in the number of elements.

But we can add a further improvement with just a small change. If you notice, we are repeatedly swapping the same element with its current parent; this element, the one on which `bubbleUp` is initially called, keeps moving towards the root: you can see in figure 2.6 that it "*bubbles up*", like a soap bubble floating towards the top of the heap. Each swap requires 3 assignments (and the use of a temporary variable), so the naïve implementation in listing 2.2 would require (at most) `3*H` assignments.

While both indices, current and parent node's, change at iteration, the value of the current element is always the same.

Figure 2.7 highlights the path that such an element has to travel (in the worst case up to the root, but possibly, as shown in the picture, bubbling up could stop at an intermediate node). It turns out that bubbling up an element in the path `P` to the heap's root is equivalent to inserting that element in the (sub)array obtained by only considering those elements in `P` (see figure 2.7.B).